

AI Student Support Assistant

Agentic AI Mini Project Report

Use Case Category	AI Student Support Assistant
Key Agent Capabilities	RAG + Tools + Memory
Domain	Agentic AI / Conversational Systems
Tools Used	Python, VS Code, Streamlit, venv

Abstract

This report presents the AI Student Support Assistant, an agentic conversational system built to answer college-related questions drawn from regulations, syllabus content, FAQs, and notices. The assistant combines conversation memory with keyword-based intent handling to respond instantly to common student queries, and is designed to be extended with a Retrieval-Augmented Generation (RAG) pipeline for broader, more accurate knowledge coverage. The report covers the project's objectives, the working of the proposed agentic solution, the implementation details and results, and the challenges and future scope identified during development.

Table of Contents

1. Project Overview	3
2. Objectives & Proposed Solution	4
3. Implementation & Results	5
4. Conclusion & Future Scope	6

1. Project Overview

Project Title

AI Student Support Assistant

Problem Statement

Students frequently need quick answers to routine college-related queries such as attendance requirements, syllabus details, FAQs, and notices. Currently, they must manually search through handbooks, circulars, or wait for faculty/staff responses, which is time-consuming and inefficient. There is a need for an intelligent assistant that can instantly respond to such queries.

Brief Description of the Project

The AI Student Support Assistant is a conversational agent designed to answer college-related questions drawn from regulations, syllabus content, FAQs, and notices. It combines Retrieval-Augmented Generation (RAG), tool-calling, and conversation memory so that it can hold context across a conversation while fetching accurate, relevant information for each student query. The assistant is built to act as a first point of contact for students, reducing dependency on manually browsing handbooks or waiting on faculty and administrative staff for routine information.

Scope of the Project

The current implementation focuses on a command-line prototype that demonstrates the core agentic building blocks — memory and rule-based tool responses — for a small set of frequently asked categories such as greetings, attendance, and syllabus queries. The scope is intentionally kept narrow at this stage so the underlying architecture (query intake, memory logging, and response generation) can be validated before it is scaled up with a full RAG pipeline, a larger intent set, and a web-based interface.

Target Users

- College students looking for quick answers on attendance rules, syllabus content, and notices.
- Faculty and administrative staff, who benefit from reduced repetitive queries handled manually.
- Institutions seeking a low-cost, extensible first step toward an AI-based student helpdesk.

Why an Agentic Approach

A simple FAQ page or static document cannot hold context or route a query intelligently. An agentic approach — where the system remembers prior turns in the conversation and can call the right "tool" or response handler for a given intent — allows the assistant to feel more like a helpful assistant than a search box, and gives a clear path to add retrieval and additional tools later without redesigning the core flow.

2. Objectives & Proposed Solution

Project Objectives

- To design a conversational assistant that reduces the time students spend searching for college information.
- To build a memory-enabled chatbot that retains context across a conversation.
- To implement keyword and intent-based query handling for common student questions.
- To lay the foundation for integrating Retrieval-Augmented Generation (RAG) for broader knowledge coverage.

How the Agentic AI Solution Works

The assistant follows an agentic workflow built around three core capabilities: Retrieval (RAG), Tools, and Memory. When a student sends a query, the input is normalized and matched against known intents (for example, greetings, attendance, or syllabus queries). Each interaction is stored in a conversation memory list so the assistant can track dialogue history. In its current stage, the assistant uses keyword-based matching to route queries to the appropriate response; the architecture is designed so this can be extended with a RAG pipeline that retrieves answers from a knowledge base of college documents (regulations, syllabus, FAQs) instead of hardcoded responses.

Key Features

- Conversational memory that stores the ongoing dialogue between the student and the assistant.
- Keyword/intent detection for common categories such as greetings, attendance, and syllabus.
- Instant, rule-based responses for frequently asked college-related questions.
- Modular function design (`save_memory`, `get_response`) that allows easy extension with new intents or a RAG layer.
- Runs as a lightweight command-line assistant, with Streamlit available for a future web-based interface.

3. Implementation & Results

Technologies / Tools Used

- Python 3 — core programming language for the assistant logic.
- Visual Studio Code — development environment used to build and test the project.
- Virtual environment (venv) — for isolated dependency management.
- Streamlit (planned/available) — for building an interactive web-based chat interface.

Working Process

The project (AI-Student-Assistant) is structured around a simple Python script, `app.py`. A `conversation_memory` list stores each exchanged message via a `save_memory()` function. The `get_response()` function receives a student's question, converts it to lowercase, and checks it against a set of keyword conditions. For instance, if the query contains "hello" or "hi", the assistant returns a welcome message; if it contains "attendance", it returns information about checking college attendance regulations; and if it contains "syllabus", it returns the relevant syllabus-related response. The assistant runs in a terminal loop, continuously accepting student input until the session is ended.

Screenshots / Output

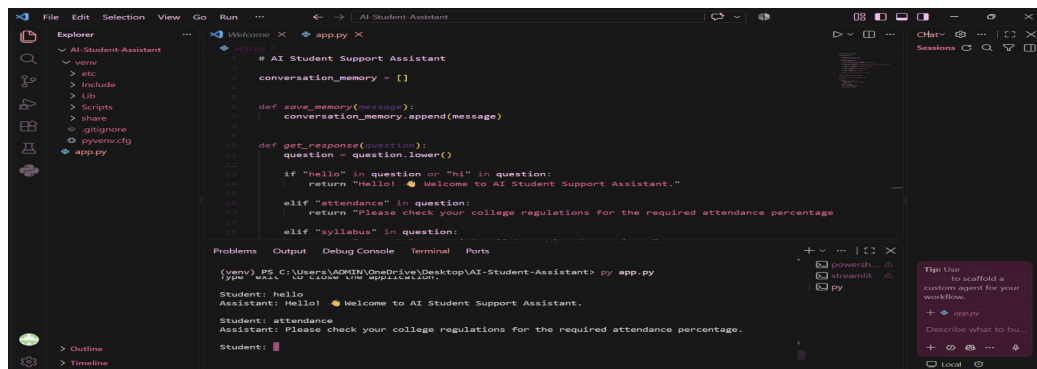
The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'AI Student Assistant' with a file named 'app.py'. The main editor area displays the code for 'app.py', which includes a 'conversation_memory' list, a 'save_memory' function, and a 'get_response' function that handles keywords like 'hello', 'hi', 'attendance', and 'syllabus'. The bottom panel shows the 'Terminal' output, which displays the execution of 'app.py' and the assistant's responses to sample queries: 'Student: hello' leads to 'Assistant: Hello! Welcome to AI Student Support Assistant.', and 'Student: attendance' leads to 'Assistant: Please check your college regulations for the required attendance percentage.' A 'Problems' pane is also visible, showing no errors.

Fig. 1 — Terminal output of the AI Student Support Assistant responding to sample queries.

Results Achieved

- The assistant successfully greets students and responds appropriately to a "hello"/"hi" input.
- It correctly identifies attendance-related queries and returns the relevant regulation guidance.
- It correctly identifies syllabus-related queries and returns the relevant response.
- Conversation history is retained in memory for the duration of the session, demonstrating basic context retention.

4. Conclusion & Future Scope

Project Conclusion

The AI Student Support Assistant demonstrates a working prototype of an agentic system that combines memory and rule-based tool logic to answer common student queries about college regulations, attendance, and syllabus. It validates the core interaction flow — capturing a query, retaining memory, and returning a relevant response — that forms the foundation for a more advanced, RAG-powered assistant.

Challenges Faced

- Handling a wide variety of ways students might phrase the same question using only keyword matching.
- Designing a memory structure that stays lightweight while still preserving conversational context.
- Structuring the codebase so new intents and a future RAG/knowledge-base layer can be added without major rework.

Future Enhancements

- Integrate a Retrieval-Augmented Generation (RAG) pipeline over actual college documents (regulations, syllabus, notices, FAQs) for more accurate and scalable answers.
- Replace keyword matching with an LLM-based intent classifier for more natural language understanding.
- Persist conversation memory across sessions instead of resetting on each run.
- Build out the Streamlit interface for a user-friendly, web-based chat experience.
- Add tool integrations such as fetching live notices or timetable data from college systems.

References

- Python 3 Documentation — <https://docs.python.org/3/>
- Streamlit Documentation — <https://docs.streamlit.io/>
- Course/workshop materials on Agentic AI use cases (RAG, Tools, Memory).